Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

КУРСОВАЯ РАБОТА

Разработка учебной системы программирования

Компилятор с ЯВУ

Дисциплина: «Системы программирования»

Вариант № 11

Исполнители:
Студенты группы 5140904/50102 Сырбу Г. В.
Федерова С. К.

Руководитель: Расторгуев В. Я.

Санкт-Петербург
2026

Оглавление

1. Задание на курсовую работу.....3
2. Постановка задачи модификации макета в части компилятора с ЯВУ..5
3. Модификация состава и структуры БД макета компилятора с ЯВУ.....8
3.1. Модификация БД лексического анализатора.....8
3.2. Модификация БД синтаксического анализатора.....9
4. Модификация алгоритма макета компилятора с ЯВУ.....18
5. Результаты экспериментальной проверки модифицированного компилятора с ЯВУ.....19
6. Выводы по результатам работы.....19

1. Задание на курсовую работу

Задание на КР дано в следующей общей формулировке:

«Необходимо выполнить доработку элементов макета учебной системы программирования до уровня, позволяющего обрабатывать «новые» для макета конструкции языка высокого уровня, примененные в соответствующем варианте».

Общая формулировка персонализирована в варианте № 11, приведенном на рис. 1.

```
EX11: PROC OPTIONS ( MAIN );
DCL A BIN FIXED INIT ( 3 );
DCL B DEC FIXED INIT ( 3 );
DCL C BIN FIXED;
C = A = ( B * 5 );
END EX11;
```

Рис. 1. Персональный вариант № 11

По договорённости с преподавателем в персональный вариант были внесены изменения, касающиеся использования скобок в арифметических выражениях: вложенное выражение со скобками было разделено на два последовательных оператора присваивания.

В результате была сформулирована следующая входная программа (рис. 2):

```
EX11: PROC OPTIONS ( MAIN );
DCL A BIN FIXED INIT ( 3 );
DCL B DEC FIXED INIT ( 3 );
DCL C BIN FIXED;

C = B * 5;

C = A = C;

END EX11;
```

Рис. 2. Модифицированный персональный вариант № 11 (согласованный с преподавателем)

Схема функционирования макета учебной системы программирования, являющегося, согласно заданию на КР, одним из элементов исходных данных, приведена на рис. 3.

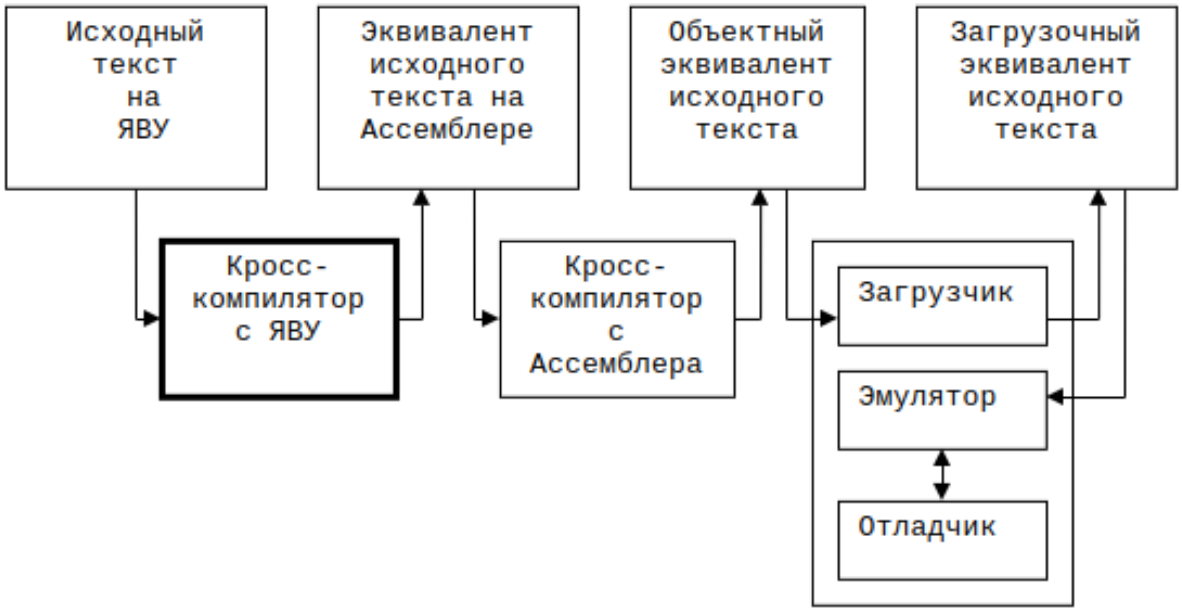

Рис. 3. Схема функционирования учебной системы программирования

Дополнительные требования к КР, вытекающие из рис.2, предполагают то, что данная система программирования работает на технологической ЭВМ (IBM PC) и является по существу кросс-системой для объектной ЭВМ (IBM 370). В этой системе:

- в качестве языка высокого уровня (ЯВУ) выбран язык, образованный из подмножества языковых конструкций ПЛ1, а исходная программа готовится в виде текстового файла технологической ЭВМ с расширением *.pli
- язык АССЕМБЛЕРА сформирован из языковых конструкций АССЕМБЛЕРА IBM 370, а ассемблеровский эквивалент исходной программы формируется в виде текстового файла технологической ЭВМ с расширением *.ass
- объектный эквивалент исходной программы готовится в формате объектных файлов операционной системы ОС IBM 370 и хранится в виде двоичного файла технологической ЭВМ с расширением *.tex

- загрузочный эквивалент исходной программы представляет собой машинный код IBM 370, запоминаемый в области ОЗУ технологической ЭВМ, являющейся зоной загрузки для эмулятора объектной ЭВМ

На рис. 2 жирным контуром выделен компилятор с ЯВУ, являющийся тем элементом макета, результаты модификации которого под персональный вариант No XX (см. рис 1) приведены в данном отчете.

2. Постановка задачи модификации макета в части компилятора с ЯВУ

Схема функционирования модифицированного макетного компилятора с ЯВУ должна соответствовать рис. 4.

Рис. 4. Схема функционирования модифицированного макетного компилятора с ЯВУ

Функционал модифицированного макетного компилятора должен обеспечить компиляцию исходного текста на ЯВУ (см. рис. 1) в эквивалент исходного текста на Ассемблере (см. рис. 5)

Метка	КОП	Операнды	Пояснение
EX11	START	0	Program start. The relative address counter is initialized to 0.
	BALR	@RBASE,0	Load the current program address into the base register @RBASE.
	USING	*,@RBASE	Declare @RBASE as the base register for addressing program objects.

Метка	КОП	Операнды	Пояснение
	ZAP	@BUF(8),B(3)	Copy the packed decimal value of B into the working buffer @BUF.
	MP	@BUF(8),@FIVE(1)	Multiply the decimal value in @BUF by the constant 5.
	CVB	@R1,@BUF	Convert the packed decimal result in @BUF to binary and place it in register @R1.
	STH	@R1, C	Store the binary result from register @R1 into variable C.
	LH	@R2,A	Load the binary value of variable A into register @R2.
	CH	@R2,C	Compare the value in register @R2 with the value stored in C.
	BC	8, @TRUE	Branch to label @TRUE if the values are equal.
	LA	@R3,0	Load value 0 into register @R3 (boolean FALSE).
	B	@SAVE	Unconditional branch to label @SAVE.
@TRUE	LA	@R3,1	Load value 1 into register @R3 (boolean TRUE).
@SAVE	STH	@R3,C	Store the boolean result (0 or 1) into variable C.
	BCR	15,@RVIX	Return control to the calling program using the return register.
A	DC	H'3'	Declare binary variable A (halfword)

Метка	КОП	Операнды	Пояснение
			initialized with value 3.
B	DC	PL3'3'	Declare decimal variable B in packed decimal format with value 3.
C	DS	H	Reserve storage for binary variable C (halfword).
@FIVE	DC	PL1'5'	Declare packed decimal constant with value 5.
	DS	0F	Align the next data item to a fullword boundary.
@BUF	DC	PL8'0'	Working buffer used for packed decimal arithmetic operations.
@RBASE	EQU	5	Assign register number 5 as the base register.
@R1	EQU	6	Register used for decimal-to-binary conversion.
@R2	EQU	2	Register used for comparison operations.
@R3	EQU	3	Register used to store the boolean result.
@RVIX	EQU	14	Register used to return control to the operating system.
	END		End of the assembly program.

Рис. 5. Эталон ассемблерного эквивалента персонального варианта № 11

3. Модификация состава и структуры БД макета компилятора с ЯВУ

3.1. Модификация БД лексического анализатора

БД лексического анализатора определяется путем выделения подмножества терминальных символов, в которое входят все терминальные символы, являющиеся разделителями термов языка.

Для персонального варианта (см. рис. 1) разделителями термов являются терминальные символы из множества:

{":", "(", ")", ";", "+", "-", "=", "_", **"*"**}

где модификация лексических настроек демо-примера состоит в расширении списка символов-разделителей термов дополнительными символами звездочка “*”

Т.к. авторы макета лексического анализатора разместили подмножество символов-разделителей термов прямо в программе лексического анализатора compress_ISXTXT(), а не в специальной таблице, то коррекция БД лексического анализатора выполнена в теле этой программы:

```
void compress_ISXTXT()
{
  if (
    ( ISXTXT[I1][I2] == '+' ||
      ISXTXT[I1][I2] == '−' ||
      ISXTXT[I1][I2] == '=' ||
      ISXTXT[I1][I2] == '(' ||
      ISXTXT[I1][I2] == ')' ||
      ISXTXT[I1][I2] == '*' /* <− добавлен разделитель '*' */
    )
    && PREDSYM == ' '
  )
  { I3−−; goto L1; }

  if ( ISXTXT[I1][I2] == ' ' &&
```

Рис 6. Фрагмент compress_ISXTXT() с добавленным символом «*»

3.2. Модификация БД синтаксического анализатора

Высокоуровневое определение модифицированного синтаксиса для персонального варианта (см. рис. 1) с выделением модификаций жирным шрифтом в определение нетерминала <ZNK> приведен ниже:

1. ⟨PRO⟩ ::= ⟨OPR⟩⟨TEL⟩⟨OEN⟩
2. ⟨OPR⟩ ::= ⟨IPR⟩:PROC_OPTIONS(MAIN);
3. ⟨IPR⟩ ::= ⟨IDE⟩
4. ⟨IDE⟩ ::= ⟨BUK⟩ | ⟨IDE⟩⟨BUK⟩ | ⟨IDE⟩⟨CIF⟩
5. ⟨BUK⟩ ::= A | B | C | D | E | M | P | X
6. ⟨CIF⟩ ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
7. ⟨TEL⟩ ::= ⟨ODC⟩ | ⟨TEL⟩⟨ODC⟩ | ⟨TEL⟩⟨OPA⟩ | **⟨TEL⟩⟨OPL⟩**
8. ⟨ODC⟩ ::= DCL_⟨IPE⟩_BIN_FIXED(⟨RZR⟩);
9. | DCL_⟨IPE⟩_BIN_FIXED(⟨RZR⟩)INIT(⟨LIT⟩);

   | DCL_⟨IPE⟩_BIN_FIXED;

   | DCL_⟨IPE⟩_BIN_FIXED_INIT(⟨LID⟩);

   | DCL_⟨IPE⟩_DEC_FIXED_INIT(⟨LID⟩);
10. ⟨IPE⟩ ::= ⟨IDE⟩
11. ⟨RZR⟩ ::= ⟨CIF⟩ | ⟨RZR⟩⟨CIF⟩
12. ⟨LIT⟩ ::= ⟨MAN⟩B
13. ⟨MAN⟩ ::= 1 | ⟨MAN⟩0 | ⟨MAN⟩1

14. **⟨LID⟩ ::= ⟨CIF⟩ | ⟨LID⟩⟨CIF⟩**

15. ⟨OPA⟩ ::= ⟨IPE⟩=⟨AVI⟩;

16. ⟨AVI⟩ ::= ⟨LIT⟩ | **⟨LID⟩** | ⟨IPE⟩| ⟨AVI⟩⟨ZNK⟩⟨LIT⟩ | **⟨AVI⟩⟨ZNK⟩⟨LID⟩** | ⟨AVI⟩⟨ZNK⟩⟨IPE⟩

17. **⟨AVL⟩ ::= ⟨AVI⟩⟨ZNL⟩⟨LIT⟩ | ⟨AVI⟩⟨ZNL⟩⟨LID⟩ | ⟨AVI⟩⟨ZNL⟩⟨IPE⟩**

18. **⟨OPL⟩ ::= ⟨IPE⟩=⟨AVL⟩;**

19. ⟨ZNK⟩ ::= + | - | ***

20. **⟨ZNL⟩ ::= =**

21. ⟨OEN⟩ ::= END_⟨IPR⟩

Здесь использованы следующие метасимволы и символы:

"<" и ">" - левый и правый ограничители нетерминального символа,

"::=" - метасимвол со смыслом "равно по определению",

"|" - метасимвол альтернативного определения "или",

"_" - терминальный символ со смыслом "пробел",

"<PRO>" - нетерминал "программа",

"<OPR>" - нетерминал "оператор пролога программы",

"<IPR>" - нетерминал "имя программы",

"<IDE>" - нетерминал "идентификатор",

"<BUK>" - нетерминал "буква",

"<CIF>" - нетерминал "цифра",

"<TEL>" - нетерминал "тело программы",

"<ODC>" - нетерминал "оператор declare",

"<IPE>" - нетерминал "имя переменной",

"<RZR>" - нетерминал "разрядность",

"<LIT>" - нетерминал "литерал",

"<MAN>" - нетерминал "мантисса",

"<LID>" — нетерминал "десятичный целочисленный литерал"

"<OPA>" - нетерминал "оператор присваивания арифметический",

"<OPL>" — нетерминал "оператор присваивания с логическим выражением"

"<AVI>" - нетерминал "арифметическое выражение",

"<AVL>" — нетерминал "логическое выражение"

"<ZNK>" - нетерминал "знак",

"<ZNL>" — нетерминал "знак логической операции"

"<OEN>" - нетерминал "оператор эпилога программы".

Сначала перепишем каждое правило грамматики, заменяя формат метаязыка Бэкуса на формат продукций (или на форму распознавания).

Для этого в каждой альтернативе каждого правила в формате метаязыка Бэкуса поменяем местами правую и левую части, изменив при этом символ "::=" на "—>". Получим следующий набор продукций:

1. <OPR><TEL><OEN> —> <PRO>
2. <IPR>:PROC_OPTIONS(MAIN); —> <OPR>
3. <IDE> —> <IPR>
4. <BUK> —> <IDE>

5. <IDE><BUK> —> <IDE>
6. <IDE><CIF> —> <IDE>
7. A —> <BUK>
8. B —> <BUK>
9. C —> <BUK>
10. D —> <BUK>
11. E —> <BUK>
12. M —> <BUK>
13. P —> <BUK>
14. X —> <BUK>
15. 0 —> <CIF>
16. 1 —> <CIF>
17. 2 —> <CIF>
18. 3 —> <CIF>
19. 4 —> <CIF>
20. 5 —> <CIF>
21. 6 —> <CIF>
22. 7 —> <CIF>
23. 8 —> <CIF>
24. 9 —> <CIF>
25. <ODC> —> <TEL>
26. <TEL><ODC> —> <TEL>
27. <TEL><OPA> —> <TEL>
28. <TEL><OPL> —> <TEL>
29. DCL_<IPE>_BIN_FIXED(<RZR>); —> <ODC>
30. DCL_<IPE>_BIN_FIXED(<RZR>)INIT(<LIT>); —> <ODC>
31. DCL_<IPE>_BIN_FIXED; —> <ODC>
32. DCL_<IPE>_BIN_FIXED_INIT(<LID>); —> <ODC>

33. DCL_<IPE>_DEC_FIXED_INIT(<LID>); —> <ODC>
34. <IDE> —> <IPE>
35. <CIF> —> <RZR>
36. <RZR><CIF> —> <RZR>
37. <MAN>B —> <LIT>
38. 1 —> <MAN>
39. <MAN>0 —> <MAN>
40. <MAN>1 —> <MAN>
41. <CIF> —> <LID>
42. <LID><CIF> —> <LID>
43. <IPE>=<AVI>; —> <OPA>
44. <LIT> —> <AVI>
45. <LID> —> <AVI>
46. <IPE> —> <AVI>
47. <AVI><ZNK><LIT> —> <AVI>
48. <AVI><ZNK><LID> —> <AVI>
49. <AVI><ZNK><IPE> —> <AVI>
50. <AVI><ZNL><LIT> —> <AVL>
51. <AVI><ZNL><LID> —> <AVL>
52. <AVI><ZNL><IPE> —> <AVL>
53. <IPE>=<AVL>; —> <OPL>
54. + —> <ZNK>
55. - —> <ZNK>
56. * —> <ZNK>
57. = —> <ZNL>
58. END_<IPR> —> <OEN>

Теперь, просматривая каждую из продукций слева-направо, сгруппируем продукции, имеющие общие части в "кусты", в которых

роль "ствола" играют общие части продукций, а роль "ветвей" – различающиеся части продукций. Получим следующий результат:

1. Куст идентификаторов и букв:

- A, B, C, D, E, M, P, X —> <BUK>
- <BUK> —> <IDE>
- <IDE>
    - <BUK> —> <IDE>
    - <CIF> —> <IDE>
- <IDE> —> <IPE>
- <IDE> —> <IPR>

2. Куст чисел и литералов:

- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 —> <CIF>
- 1 —> <MAN>
- <MAN>
    - B —> <LIT>
    - 0 —> <MAN>
    - 1 —> <MAN>
- <CIF>
    - —> <RZR>
    - —> <LID>
- <RZR><CIF> —> <RZR>
- <LID><CIF> —> <LID>

3. Куст оператора объявления (DCL):

- DCL_ <IPE>_
    - BIN_FIXED
        - (<RZR>)
            - ; —> <ODC>

            - INIT(<LIT>); —> <ODC>
        - ; —> <ODC>
        - _INIT(<LID>); —> <ODC>
    - DEC_FIXED_INIT(<LID>); —> <ODC>

4. Куст выражений (арифметических и логических):
    - <LIT> —> <AVI>
    - <LID> —> <AVI>
    - <IPE> —> <AVI>
    - <AVI>
        - <ZNK>
            - <LIT> —> <AVI>
            - <LID> —> <AVI>
            - <IPE> —> <AVI>
        - <ZNL>
            - <LIT> —> <AVL>
            - <LID> —> <AVL>
            - <IPE> —> <AVL>
5. Куст операторов присваивания:
    - <IPE>=
        - <AVI>; —> <OPA>
        - <AVL>; —> <OPL>
6. Куст знаков и разделителей:
    - +, -, * —> <ZNK>
    - = —> <ZNL>
7. Куст структуры программы (Тело):
    - <ODC> —> <TEL>
    - <TEL>

- <ODC> —> <TEL>
- <OPA> —> <TEL>
- <OPL> —> <TEL>

8. Куст корня программы:

- <IPR>:PROC_OPTIONS(MAIN); —> <OPR>
- <OPR><TEL><OEN> —> <PRO>
- END_<IPR> —> <OEN>

1. Низкоуровневый эквивалент синтаксиса, полученного после преобразования продукций в кусты, размещен в трех таблицах компилятора:

- SINT - таблица синтаксических правил в форме распознавания;
- VXOD - таблица входов в кусты грамматики;
- TPR - матрица непосредственной предшествуемости, на основе которой строится матрица достижимости.

Были расширены размеры таблиц и буферов:

```
#define NSINT     310
#define NVXOD      57
#define NSTROKA  4000
#define NNETRM     20
#define MAXLTXT   100
```

В таблице SINT изменены и добавлены ветви, которые реализуют:

- объявления BIN FIXED без явного указания разрядности;
- объявления BIN FIXED INIT(...) и DEC FIXED INIT(...) с десятичными литералами;
- десятичный целочисленный литерал <LID>;
- арифметическое умножение * как знак <ZNK>;

- логическое выражение <AVL> и логический оператор присваивания <OPL>;
- знак логической операции <ZNL>, которому соответствует терминал =.

Фрагменты таблицы SINT, относящиеся к модификации оператора объявления, приведены ниже:

```
/* Основная ветвь DCL ... BIN FIXED */
/* 59 */ {  60 ,   58 , "(  " ,  232 },
/* 60 */ {  61 ,   59 , "RZR" ,    0 },
/* 61 */ {  62 ,   60 , ")  " ,    0 },
/* 62 */ {  63 ,   61 , ";  " ,   65 },
/* 63 */ {  64 ,   62 , "ODC" ,    0 },
/* 64 */ {   0 ,   63 , "*  " ,    0 },
/* 65 */ {  66 ,   61 , "I  " ,    0 },
/* 66 */ {  67 ,   65 , "N  " ,    0 },
/* 67 */ {  68 ,   66 , "I  " ,    0 },
/* 68 */ {  69 ,   67 , "T  " ,    0 },
/* 69 */ {  70 ,   68 , "(  " ,    0 },
/* 70 */ {  71 ,   69 , "LIT" ,  246 },
/* 71 */ {  72 ,   70 , ")  " ,    0 },
/* 72 */ {  73 ,   71 , ";  " ,    0 },
/* 73 */ {  64 ,   72 , "ODC" ,    0 },

/* Ветвь DEC FIXED */
/* 201 */ { 202 ,   49 , "D  " ,    0 },
/* 202 */ { 203 ,  201 , "E  " ,    0 },
/* 203 */ { 204 ,  202 , "C  " ,    0 },
/* 204 */ { 205 ,  203 , "   " ,    0 },
/* 205 */ { 206 ,  204 , "F  " ,    0 },
/* 206 */ { 207 ,  205 , "I  " ,    0 },
/* 207 */ { 208 ,  206 , "X  " ,    0 },
/* 208 */ { 209 ,  207 , "E  " ,    0 },
/* 209 */ { 210 ,  208 , "D  " ,    0 },
/* 210 */ { 211 ,  209 , "(  " ,  250 },
/* 211 */ { 212 ,  210 , "RZR" ,    0 },
```

```
/* 212 */ { 213 ,  211 , ")  " ,    0 },
/* 213 */ { 214 ,  212 , ";  " ,  216 },
/* 214 */ { 215 ,  213 , "ODC" ,    0 },
/* 215 */ {   0 ,  214 , "*  " ,    0 },
```

Для объявлений без явной разрядности и для десятичных инициализаторов добавлены отдельные цепочки:

```
/* BIN FIXED INIT(<LID>) и BIN FIXED; */
/* 232 */ { 233 ,   58 , "   " , 233 },
/* 233 */ { 234 ,   58 , "I  " , 241 },
/* 234 */ { 235 ,  233 , "N  " ,   0 },
/* 235 */ { 236 ,  234 , "I  " ,   0 },
/* 236 */ { 237 ,  235 , "T  " ,   0 },
/* 237 */ { 238 ,  236 , "(  " ,   0 },
/* 238 */ { 239 ,  237 , "LID" , 240 },
/* 239 */ { 242 ,  238 , ")  " ,   0 },
/* 240 */ { 239 ,  237 , "LID" ,   0 },
/* 241 */ {  63 ,   58 , ";  " ,   0 },
/* 242 */ {  63 ,  239 , ";  " ,   0 },
/* 246 */ {  71 ,   69 , "RZR" ,   0 },

/* DEC FIXED INIT(<LID>) */
/* 250 */ { 251 ,  209 , "   " , 251 },
/* 251 */ { 252 ,  209 , "I  " , 257 },
/* 252 */ { 253 ,  251 , "N  " ,   0 },
/* 253 */ { 254 ,  252 , "I  " ,   0 },
/* 254 */ { 255 ,  253 , "T  " ,   0 },
/* 255 */ { 256 ,  254 , "(  " ,   0 },
/* 256 */ { 259 ,  255 , "LID" , 258 },
/* 257 */ { 214 ,  209 , ";  " ,   0 },
/* 258 */ { 259 ,  255 , "LID" ,   0 },
/* 259 */ { 260 ,  256 , ")  " ,   0 },
/* 260 */ { 214 ,  259 , ";  " ,   0 },
/* 265 */ { 222 ,  220 , "LID" ,   0 },
```

Нетерминалы <LID>, <ZNL>, <AVL> и <OPL> размещены в таблице SINT отдельными структурными цепочками:

```
/* TEL -> TEL OPL */
/* 270 */ { 271 ,  179 , "OPL" ,   0 },
/* 271 */ { 272 ,  270 , "TEL" ,   0 },
/* 272 */ {   0 ,  271 , "*  " ,   0 },

/* LID -> LID CIF */
/* 273 */ { 274 ,    0 , "LID" ,   0 },
/* 274 */ { 275 ,  273 , "CIF" ,   0 },
/* 275 */ { 276 ,  274 , "LID" ,   0 },
/* 276 */ {   0 ,  275 , "*  " ,   0 },

/* ZNL -> = */
/* 280 */ { 281 ,    0 , "=  " ,   0 },
/* 281 */ { 282 ,  280 , "ZNL" ,   0 },
/* 282 */ {   0 ,  281 , "*  " ,   0 },

/* AVL -> AVI ZNL (LIT | LID | IPE) */
/* 283 */ { 284 ,  164 , "ZNL" ,   0 },
/* 284 */ { 285 ,  283 , "LIT" , 288 },
/* 285 */ { 286 ,  284 , "AVL" ,   0 },
/* 286 */ {   0 ,  285 , "*  " ,   0 },
/* 288 */ { 289 ,  283 , "LID" , 291 },
/* 289 */ {   0 ,  288 , "AVL" ,   0 },
/* 291 */ { 292 ,  283 , "IPE" ,   0 },
/* 292 */ {   0 ,  291 , "AVL" ,   0 },

/* OPL -> IPE = AVL ; */
/* 294 */ { 295 ,    0 , "IPE" ,   0 },
/* 295 */ { 296 ,  294 , "=  " ,   0 },
/* 296 */ { 297 ,  295 , "AVL" ,   0 },
/* 297 */ { 298 ,  296 , ";  " ,   0 },
/* 298 */ {   0 ,  297 , "OPL" ,   0 },
```

```
/* Дополнительные входы из CIF и OPA */
/* 300 */ { 301 ,  136 , "LID" ,   0 },
/* 301 */ {   0 ,  300 , "*  " ,   0 },
/* 302 */ { 303 ,  154 , "AVL" ,   0 },
/* 303 */ { 304 ,  302 , ";  " ,   0 },
/* 304 */ { 305 ,  303 , "OPL" ,   0 },
/* 305 */ {   0 ,  304 , "*  " ,   0 },
```

Таблица входов VXOD расширена четырьмя новыми нетерминалами.

```
struct
 {
  char SYM [4];
  int  VX;
  char TYP;
 } VXOD [ NVXOD ] =
{
  {/*.   1     .*/   "AVI" , 164 , 'N' },
  {/*.   2     .*/   "BUK" ,  93 , 'N' },
  {/*.   3     .*/   "CIF" , 136 , 'N' },
  {/*.   4     .*/   "IDE" ,  96 , 'N' },
  {/*.   5     .*/   "IPE" , 153 , 'N' },
  {/*.   6     .*/   "IPR" , 113 , 'N' },
  {/*.   7     .*/   "LIT" , 161 , 'N' },
  {/*.   8     .*/   "MAN" , 143 , 'N' },
  {/*.   9     .*/   "ODC" , 176 , 'N' },
  {/*.  10     .*/   "OEN" ,   0 , 'N' },
  {/*.  11     .*/   "OPA" ,   0 , 'N' },
  {/*.  12     .*/   "OPR" , 171 , 'N' },
  {/*.  13     .*/   "PRO" ,   0 , 'N' },
  {/*.  14     .*/   "RZR" , 139 , 'N' },
  {/*.  15     .*/   "TEL" , 179 , 'N' },
  {/*.  16     .*/   "ZNK" ,   0 , 'N' },
  {/*.  17     .*/   "LID" , 273 , 'N' },
  {/*.  18     .*/   "AVL" , 283 , 'N' },
  {/*.  19     .*/   "OPL" , 294 , 'N' },
```

```
  {/*.  20     .*/   "ZNL" , 280 , 'N' },
  ...
  {/*.  54     .*/   "=  " , 280 , 'T' },
  {/*.  56     .*/   "*  " , 198 , 'T' }
};
```

Матрица TPR расширена с 16 до 20 столбцов, так как к множеству нетерминалов добавлены LID, AVL, OPL и ZNL.

```
/*       AVI ... ZNK LID AVL OPL ZNL */
/*AVI*/  1  ...  0   0   1   0   0
/*CIF*/  0  ...  0   1   0   0   0
/*IPE*/  1  ...  0   0   0   1   0
/*LID*/  1  ...  0   1   0   0   0
/*OPL*/  0  ...  0   0   0   0   0  /* с достижением TEL
*/
/*=*/    0  ...  0   0   0   0   1
/***/    0  ...  1   0   0   0   0
```

Таким образом, в таблицах БД синтаксического анализатора символ * используется как арифметический знак <ZNK>, а символ = в правой части выражения используется как знак логической операции <ZNL>. Это соответствует высокоуровневому описанию:

```
<ZNK> ::= + | - | *
<ZNL> ::= =
<AVL> ::= <AVI><ZNL><LIT> | <AVI><ZNL><LID> |
<AVI><ZNL><IPE>
<OPL> ::= <IPE>=<AVL>;
```

4. Модификация алгоритма макета компилятора с ЯВУ

Модификация алгоритма макета компилятора с ЯВУ была произведена параметрически путем модификации той части параметров настройки компилятора ЯВУ, которая относится к семантике ЯВУ. Этой частью

семантических параметров является библиотека семантических подпрограмм.

Объем модификаций этой библиотеки определяется объемом модификации синтаксиса ЯВУ. Для персонального варианта N 11 модификация синтаксиса привела к необходимости:

- изменить синтаксический анализатор для выбора ветви <OPL> при разборе оператора вида C = A = C;
- изменить обработку объявлений в ODC1();
- изменить генерацию арифметического выражения в AVI2();
- добавить семантические подпрограммы для новых нетерминалов LID, AVL, OPL, ZNL;
- изменить завершение программы в OEN2(), чтобы формировались объявления двоичных и десятичных переменных, packed decimal констант, рабочего буфера и регистров.

Для выбора логического оператора присваивания добавлены две вспомогательные функции просмотра вперед. Они используются синтаксическим анализатором при выборе альтернативы:

```
int is_logical_assignment ( int pos )
 {
  int eq_count = 0;
  int p = pos;

  while ( p < NSTROKA && STROKA [p] != '\x0' && STROKA
[p] != ';' )
   {
    if ( STROKA [p] == '=' )
      eq_count++;
    p++;
   }

  return eq_count > 1;
 }
```

```
int has_equal_before_semicolon ( int pos )
 {
  int p = pos;

  while ( p < NSTROKA && STROKA [p] != '\x0' && STROKA
[p] != ';' )
   {
    if ( STROKA [p] == '=' )
      return 1;
    p++;
   }

  return 0;
 }
```

В программе sint_ANAL() эти функции применяются в точке выбора альтернатив:

```
if ( !strcmp ( SINT[J].DER, "OPA" ) &&
     SINT[J].ALT != 0 &&
     is_logical_assignment ( I ) )
  J = SINT[J].ALT;

if ( !strcmp ( SINT[J].DER, "AVI" ) &&
     SINT[J].ALT != 0 &&
     K > 0 &&
     !strcmp ( CEL[K-1].CEL1, "OPL" ) &&
     has_equal_before_semicolon ( I ) )
  J = SINT[J].ALT;
```

Данные изменения обеспечивают строгий маршрут разбора оператора C = A = C; через нетерминалы <OPL>, <AVL> и <ZNL>, а не через арифметический знак <ZNK>.

Семантическая подпрограмма ODC1() изменена так, чтобы заполнять таблицу имен SYM для двоичных и десятичных переменных, поддерживать отсутствие явной разрядности и сохранять десятичную инициализацию:

```
int ODC1 ()
 {
  int i, cur;
  FORM ();
  if (IFORMT < 3) return 2;

  for ( i = 0; i < ISYM; i++ )
   if ( !strcmp ( SYM [i].NAME, FORMT [1] ) ) return 6;

  strcpy ( SYM [ISYM].NAME, FORMT [1] );

  if ( !strcmp ( FORMT [2], "BIN" ) ) SYM [ISYM].TYPE =
'B';
  else if ( !strcmp ( FORMT [2], "DEC" ) ) SYM
[ISYM].TYPE = 'D';
  else return 2;

  if ( IFORMT > 4 && isdigit(FORMT[4][0]) )
   {
    strncpy ( SYM [ISYM].RAZR, FORMT [4], 4 );
    SYM [ISYM].RAZR[4] = '\0';
    cur = 5;
   }
  else
   {
    strcpy ( SYM [ISYM].RAZR, (SYM[ISYM].TYPE == 'B') ?
"15" : "3" );
    cur = 4;
   }
```

```
  if ( cur < IFORMT && !strcmp ( FORMT [cur], "INIT" ) )
   {
    if (cur + 1 < IFORMT)
      strcpy ( SYM [ISYM].INIT, FORMT [cur + 1] );
    else
      SYM [ISYM].INIT[0] = '\0';
   }
  else
    SYM [ISYM].INIT[0] = '\0';

  ISYM++;
  return 0;
 }
```

В подпрограмме AVI2() добавлена генерация для:

- загрузки двоичной переменной в регистр @R2;
- копирования десятичной packed decimal переменной в буфер @BUF;
- умножения packed decimal значения на константу;
- преобразования packed decimal результата в двоичный регистр @R1.

Фрагмент генерации для выражения B * 5:

```
if ( SYM [i].TYPE == 'D' )
 {
  memcpy ( ASS_CARD._BUFCARD.OPERAC, "ZAP", 3 );
  sprintf ( opbuf, "@BUF(8),%s(%s)", FORMT[0],
SYM[i].RAZR );
  memcpy ( ASS_CARD.BUFCARD + 15, opbuf, strlen(opbuf) );
  SETCOMM ( "Copy pdec B->BUF" );
  ZKARD ();
  return 0;
 }
```

```
if ( op_sign == '*' )
 {
  char const_val[10];
  strcpy ( const_val, FORMT[IFORMT-1] );

  const_label ( clbl, const_val );
  sprintf ( opbuf, "@BUF(8),%s(1)", clbl );

  memcpy ( ASS_CARD._BUFCARD.OPERAC, "MP", 2 );
  memcpy ( ASS_CARD.BUFCARD + 15, opbuf, strlen(opbuf) );
  SETCOMM ( "Mul BUF by 5->BUF" );
  ZKARD ();

  memcpy ( ASS_CARD._BUFCARD.OPERAC, "CVB", 3 );
  strcpy ( ASS_CARD._BUFCARD.OPERAND, "@R1,@BUF" );
  SETCOMM ( "pdec BUF ->int @R1" );
  ZKARD ();

  strcpy ( RESULT_REG, "@R1" );
  return 0;
 }
```

Для логического выражения добавлена подпрограмма AVL2(). Она формирует сравнение левого значения, предварительно помещенного в @R2, с правым операндом логического выражения:

```
int AVL2 ()
 {
  FORM ();
  if ( IFORMT < 2 )
    return 2;

  avl2_emit_compare_rhs ( FORMT [IFORMT - 1] );
  return 0;
```

```
 }
```

Функция avl2_emit_compare_rhs() генерирует последовательность команд IBM 370 для получения булева результата:

```
static void avl2_emit_compare_rhs ( const char
*rhs_name )
{
  memcpy ( ASS_CARD._BUFCARD.OPERAC, "CH", 2 );
  strcpy ( ASS_CARD._BUFCARD.OPERAND, "@R2," );
  strcat ( ASS_CARD._BUFCARD.OPERAND, rhs_name );
  ZKARD ();

  memcpy ( ASS_CARD._BUFCARD.OPERAC, "BC", 2 );
  strcpy ( ASS_CARD._BUFCARD.OPERAND, "8,@TRUE" );
  ZKARD ();

  memcpy ( ASS_CARD._BUFCARD.OPERAC, "LA", 2 );
  strcpy ( ASS_CARD._BUFCARD.OPERAND, "@R3,0" );
  ZKARD ();

  memcpy ( ASS_CARD._BUFCARD.OPERAC, "B", 1 );
  strcpy ( ASS_CARD._BUFCARD.OPERAND, "@SAVE" );
  ZKARD ();

  memcpy ( ASS_CARD._BUFCARD.METKA, "@TRUE", 5 );
  memcpy ( ASS_CARD._BUFCARD.OPERAC, "LA", 2 );
  strcpy ( ASS_CARD._BUFCARD.OPERAND, "@R3,1" );
  ZKARD ();

  strcpy ( RESULT_REG, "@R3" );
  strcpy ( PENDING_LABEL, "@SAVE" );
}
```

Семантическая подпрограмма OPL2() сохраняет результат логического выражения в левую часть оператора присваивания:

```
int OPL2 ()
 {
  int i;
  FORM ();

  for ( i = 0; i < ISYM; i++ )
   {
    if ( !strcmp ( SYM [i].NAME, FORMT [0] ) &&
         strlen ( SYM [i].NAME ) == strlen ( FORMT
[0] ) )
     {
      if ( SYM [i].TYPE == 'B' )
       {
        if ( PENDING_LABEL[0] != '\0' )
         {
          strcpy ( ASS_CARD._BUFCARD.METKA, PENDING_LABEL
);
          PENDING_LABEL[0] = '\0';
         }

        memcpy ( ASS_CARD._BUFCARD.OPERAC, "STH", 3 );
        strcpy ( ASS_CARD._BUFCARD.OPERAND, RESULT_REG );
        strcat ( ASS_CARD._BUFCARD.OPERAND, "," );
        strcat ( ASS_CARD._BUFCARD.OPERAND, FORMT [0] );
        SETCOMM ( "Store bool ->C" );
        ZKARD ();
        return 0;
       }
      else
        return 3;
     }
   }
  return 4;
```

```
 }
```

Для новых нетерминалов добавлены пары семантических подпрограмм. Часть из них имеет пустую семантику, так как сами нетерминалы нужны для синтаксического распознавания:

```
int LID1 () { return 0; }
int LID2 () { return 0; }
int AVL1 () { return 0; }
int OPL1 () { return 0; }
int ZNL1 () { return 0; }
int ZNL2 () { return 0; }
```

Массив семантических подпрограмм FUN расширен до 20 строк:

```
int (* FUN [NNETRM][2]) () =
 {
   {/*    1  */    AVI1, AVI2 },
   {/*    2  */    BUK1, BUK2 },
   {/*    3  */    CIF1, CIF2 },
   {/*    4  */    IDE1, IDE2 },
   {/*    5  */    IPE1, IPE2 },
   {/*    6  */    IPR1, IPR2 },
   {/*    7  */    LIT1, LIT2 },
   {/*    8  */    MAN1, MAN2 },
   {/*    9  */    ODC1, ODC2 },
   {/*   10  */    OEN1, OEN2 },
   {/*   11  */    OPA1, OPA2 },
   {/*   12  */    OPR1, OPR2 },
   {/*   13  */    PRO1, PRO2 },
   {/*   14  */    RZR1, RZR2 },
   {/*   15  */    TEL1, TEL2 },
   {/*   16  */    ZNK1, ZNK2 },
   {/*   17  */    LID1, LID2 },
```

```
  {/*   18  */    AVL1, AVL2 },
  {/*   19  */    OPL1, OPL2 },
  {/*   20  */    ZNL1, ZNL2 }
};
```

В результате модификации алгоритма макетный компилятор формирует ассемблерный эквивалент для обоих новых фрагментов персонального варианта: арифметического выражения с десятичным операндом C = B * 5; и логического присваивания C = A = C;.

5. Результаты экспериментальной проверки модифицированного компилятора с ЯВУ

Работоспособность модифицированного под персональный вариант макета компилятора с ЯВУ была подтверждена испытательным прогоном в среде ОС Linux. В качестве входного текста использована программа:

```
EX11: PROC OPTIONS ( MAIN );
DCL A BIN FIXED INIT ( 3 );
DCL B DEC FIXED INIT ( 3 );
DCL C BIN FIXED;
C = B * 5;
C = A = C;
END EX11;
```

В результате на выходе был получен следующий текст на Ассемблере:

```
~/projects/program-systems-lab step-2-lexis !1 ?4 ❯ fold -w 80 lab-step-2/output/ex11.ass
EX11     START 0             Init rel addr=0
         BALR  @RBASE,0      Set @RBASE=cur PC
         USING *,@RBASE      Use @RBASE for addr
         ZAP   @BUF(8),B(3) Copy pdec B->BUF
         MP    @BUF(8),@FIVE(1) Mul BUF by 5->BUF
         CVB   @R1,@BUF      pdec BUF ->int @R1
         STH   @R1,C         Store int ->C
         LH    @R2,A         Load A ->@R2
         CH    @R2,C         Compare @R2 with C
         BC    8,@TRUE       Branch if equal
         LA    @R3,0         Set @R3=0 (F)
         B     @SAVE         Jump to save
@TRUE    LA    @R3,1         Set @R3=1 (T)
@SAVE    STH   @R3,C         Store bool ->C
         BCR   15,@RVIX      Return to caller
A        DC    H'3'          Store halfword init
B        DC    PL3'3'        Store pdec init
C        DS    H             Reserve halfword
@FIVE    DC    PL1'5'        Store packed const
         DS    0F            Align next item
@BUF     DC    PL8'0'        Clear packed BUF
@RBASE   EQU   5             Set @RBASE=5
@R1      EQU   6             Set @R1=6
@R2      EQU   2             Set @R2=2
@R3      EQU   3             Set @R3=3
@RVIX    EQU   14            Set @RVIX=14
         END                 Program ends
```

Полученный выходной текст совпадает с эталонным ассемблерным эквивалентом, приведенным в разделе постановки задачи на рис. 5. Совпадение подтверждает корректность выполненных модификаций лексической БД, синтаксической БД и семантических подпрограмм компилятора с ЯВУ.

6. Выводы по результатам работы

Порученное задание выполнено успешно. Успешность выполнения порученного задания подтверждена полным совпадением выходного текста, полученного при экспериментальной проверке компилятора с ЯВУ, с эталоном на рис. 5.

В ходе работы макет компилятора был доработан для обработки новых конструкций персонального варианта N 11: объявления десятичной переменной DEC FIXED INIT, арифметического выражения с операцией

умножения и логического выражения вида A = C внутри оператора присваивания. В результате модифицированный компилятор принимает входную программу персонального варианта и формирует корректный ассемблерный эквивалент.